SECTION 12 — TESTING & DEBUGGING

12.1 Testing Methods

Testing is the process of evaluating a software system to verify that it satisfies the specified requirements and to identify defects. The following testing methods were applied:

12.1.1 Unit Testing

Individual functions and API endpoints were tested in isolation using **Postman**. Each controller function (login, createStudent, payFee, etc.) was tested with valid input, invalid input, and boundary cases.

12.1.2 Integration Testing

Multiple modules were tested together to ensure they work correctly as a combined system. For example: - Student creation + Fee assignment + Payment processing was tested end-to-end. - Teacher login + Attendance marking + Report generation was tested.

12.1.3 User Acceptance Testing (UAT)

The fully integrated system was tested by simulating the actual workflow of each user role: - Admin logged in and performed student management, fee collection, and payroll processing. - Teacher logged in, marked attendance, entered grades, and viewed payslip. - Student logged in, viewed attendance, grades, fees paid, and timetable.

12.1.4 Security Testing

- Attempted access to protected routes without a token — verified 401 Unauthorized response.
- Tested Student role accessing Admin-only routes — verified 403 Forbidden response.
- Verified passwords are stored as bcryptjs hashes in MongoDB (not plain text).
- Tested rate limiter with 105 rapid requests — verified 429 Too Many Requests after 100.

12.2 Test Cases Table

Module: Authentication

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC01	Login with valid Admin credentials	admin@college.com / admin123	JWT token returned, role = admin	JWT token returned	✅ Pass
TC02	Login with wrong password	admin@college.com / wrong	401 “Invalid credentials”	401 returned	✅ Pass
TC03	Login with unregistered email	fake@test.com / pass	401 “Invalid credentials”	401 returned	✅ Pass
TC04	Access protected route without token	No Authorization header	401 “Not authorized, no token”	401 returned	✅ Pass
TC05	Access protected route with valid token	Bearer [valid token]	200 OK with data	200 OK returned	✅ Pass
TC06	Access protected route with expired token	Bearer [expired token]	401 “Token invalid or expired”	401 returned	✅ Pass

---

Module: Student Management

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC07	Add student with all valid fields	Complete student data	201 Student created	201 returned	✅ Pass
TC08	Add student with duplicate roll number	Existing rollNo value	409 “Roll number already exists”	409 returned	✅ Pass

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC09	Add student without required field (course)	Missing course field	400 Validation error	400 returned	✅ Pass
TC10	Get all students	GET /api/students	Paginated student list	List returned	✅ Pass
TC11	Get student by valid ID	Valid MongoDB ObjectId	Student detail object	Student data returned	✅ Pass
TC12	Get student by invalid ID	Random invalid ID	404 “Student not found”	404 returned	✅ Pass
TC13	Update student details	PUT with changed semester	200 Updated student	Updated correctly	✅ Pass
TC14	Delete student	Valid student ID	200 “Student deleted”	200 returned	✅ Pass
TC15	Student role accessing delete endpoint	Student JWT token	403 “not authorized”	403 returned	✅ Pass

---

Module: Fee Management

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC16	Process fee payment (Cash)	Student ID + amount + method=Cash	201 with receipt number	Receipt generated	✅ Pass
TC17	Process fee payment (UPI)	Student ID + method=UPI + ref	201 with receipt and ref	Correct	✅ Pass
TC18	Get payment history for student	Student ID	List of all payments	History returned	✅ Pass

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC19	Process payment with missing amount	No amount field	400 Validation error	400 returned	✅ Pass

---

Module: Payroll

TC#	Test Case Description	Input Data	Expected Result	Actual Result	Status
TC20	Get payroll sheet	GET /api/payroll/sheet	All employee salary records	Sheet returned	✅ Pass
TC21	Process monthly payroll	Month = Feb 2026	Salary records created for all	Records created	✅ Pass
TC22	Get individual payslip	Teacher ID	Payslip with basic, deductions, net	Payslip correct	✅ Pass

---

Module: Security

TC#	Test Case Description	Expected Result	Actual Result	Status
TC23	Attempt NoSQL injection in login body	Input sanitized; no unauthorized access	Blocked	✅ Pass
TC24	Exceed rate limit (105 requests in 15 min)	429 Too Many Requests	429 returned after 100	✅ Pass
TC25	Password stored in plain text?	DB must show bcrypt hash	bcrypt hash confirmed in Atlas	✅ Pass
TC26	CORS — request from unauthorized domain	Blocked with CORS error	CORS rejected	✅ Pass

---

12.3 Test Results Summary

Module	Test Cases	Passed	Failed	Pass Rate
Authentication	6	6	0	100%
Student Management	9	9	0	100%
Fee Management	4	4	0	100%
Payroll	3	3	0	100%
Security	4	4	0	100%
TOTAL	**26**	**26**	**0**	**100%**

12.4 Bug Fixing Process

During the development and testing phases, the following issues were identified and resolved:

Bug #	Bug Description	Module	Fix Applied
BUG-01	CORS error when frontend called backend API	Auth	Added `cors({ origin: FRONTEND_URL })` in index.js
BUG-02	JWT token not persisted after page refresh	Frontend Auth	Switched Zustand store to `persist` middleware with localStorage
BUG-03	Password comparison always returning false	Auth	Fixed bcryptjs `hash` rounds — was using plain text comparison
BUG-04	Paginated student list returning incorrect total count	Student	Added `countDocuments(filter)` with the same filter as `find()`

Bug #	Bug Description	Module	Fix Applied
BUG-05	Fee receipt number had collision on fast requests	Finance	Changed from `Math.random()` to `Date.now()` + unique suffix
BUG-06	Mongoose `populate()` returning null for teacherProfileId	User	Fixed ObjectId reference — incorrect collection name in ref field
BUG-07	React Router protected route redirecting even when logged in	Frontend	Fixed Zustand `isAuthenticated` flag initialization from persisted state
BUG-08	Dashboard stats not updating after adding new student	Dashboard	Added `useEffect` dependency on navigation events

---

12.5 Final Deployment

Deployment Steps (Render.com)

```
 1. Push code to GitHub repository

2. Backend Deployment (Render.com Web Service):
   - Connect GitHub repository
   - Set Build Command: npm install
   - Set Start Command: node src/index.js
   - Add Environment Variables:
       MONGODB_URI = [MongoDB Atlas URI]
       JWT_SECRET  = [32-char random string]
       JWT_REFRESH_SECRET = [32-char random string]
       PORT = 5000
       NODE_ENV = production
       FRONTEND_URL = [frontend URL]

3. Frontend Deployment (Render.com Static Site):
   - Connect GitHub repository
   - Set Build Command: npm install && npm run build
```

```
    - Set Publish Directory: dist
    - Add Environment Variable:
        VITE_API_URL = [backend URL]/api

 4. MongoDB Atlas Setup:
    - Create free M0 cluster
    - Set IP Whitelist to 0.0.0.0/0 (allow all)
    - Create database user
    - Copy connection URI to backend env
```

Live URLs

Service	URL
Frontend	`https://college-management-frontend.onrender.com`
Backend API	`https://college-management-backend.onrender.com`
Database	MongoDB Atlas (Singapore region)

12.6 Conclusion

The **College Management System** was successfully developed, tested, and deployed. All 26 test cases passed with a 100% pass rate. The system fulfills all functional and non-functional requirements defined in the SRS. It provides a modern, secure, and scalable digital platform that effectively replaces manual paper-based college administration operations.

REFERENCES

#	Reference
1	MongoDB, Inc. *MongoDB Manual*. https://www.mongodb.com/docs/
2	OpenJS Foundation. *Node.js v18 Documentation*. https://nodejs.org/en/docs/
3	OpenJS Foundation. *Express.js 4.x API Reference*. https://expressjs.com/en/4x/api.html
4	Meta Open Source. *React 18 Official Documentation*. https://react.dev/
5	Mongoose Team. *Mongoose ODM Documentation v7*. https://mongoosejs.com/docs/
6	JWT. *JSON Web Tokens — Introduction*. https://jwt.io/introduction
7	Vite Team. *Vite Build Tool Documentation*. https://vitejs.dev/guide/
8	Tailwind Labs. *Tailwind CSS v3 Documentation*. https://tailwindcss.com/docs/
9	Render. *Render Deployment Documentation*. https://render.com/docs
10	Pressman, R.S. (2014). *Software Engineering: A Practitioner's Approach*, 8th Ed. McGraw-Hill
11	Tanenbaum, A.S. (2016). *Modern Operating Systems*, 4th Ed. Pearson
12	Silberschatz, A., Korth, H.F. & Sudarshan, S. (2020). *Database System Concepts*, 7th Ed. McGraw-Hill

© 2026 — College Management System | MERN Stack Department of Computer Science | [Institute Name] | Academic Year 2025–2026